同济大学计算机系

操作系统课程实验报告

学　　号 2353761

姓　　名 王皙晶

专　　业 计算机科学与技术

授课老师 方钰

实验五：UNIX V6++完整的进程图象

一、实验目的

（1）在 UNIX V6++中编译和运行调试一个简单的 C++代码。
（2）通过查找关键地址单元的值，绘制出整个进程的图象。
（3）加深对 UNIX 进程图象的理解，特别是对逻辑地址空间与物理地址空间的理解。

二、实验结果

（1）完成实验 4.1~4.2，获取现运行进程 User 结构、Proc 结构和 Text 结构的内容，截图或绘制表格说明，并总结在 UNIX V6++中获取进程的代码段和可交换部分起始位置的逻辑地址和物理地址的方法。

- **User 结构**

```
> u_procp = 0xc011fb20 <g_ProcessManager+128>
v u_MemoryDescriptor
    USER_SPACE_SIZE = 8388608
    USER_SPACE_PAGE_TABLE_CNT = 2
    USER_SPACE_START_ADDRESS = 0
  v m_UserPageTableArray = 0xc0208000
      ENTRY_CNT_PER_PAGETABLE = 1024
      SIZE_PER_PAGETABLE_MAP = 4194304
    > m_Entrys
    m_TextStartAddress = 4198400
    m_TextSize = 12288
    m_DataStartAddress = 4210688
    m_DataSize = 20480
    m_StackSize = 4096
```

变量名称		含义	值
Process* u_procp		Proc结构的逻辑地址	0xc011fb20
MemoryDescriptor u_MemoryDescriptor（定义如下，此处均为逻辑地址）			
PageTable*	m_UserPageTableArray	相对映射表首地址	0xc0208000
unsigned long	m_TextStartAddress	代码段起始地址	4198400(0x401000)=4M+4K
unsigned long	m_TextSize	代码段长度	12288(0x3000)=12K
unsigned long	m_DataStartAddress	数据段起始地址	4210688(0x404000)=4M+16K
unsigned long	m_DataSize	数据段长度	20480(0x5000)=20K
unsigned long	m_StackSize	栈段长度	4096(0x1000)=4K

- Proc 结构

```
u_procp = 0xc011fb20 <g_ProcessManager+128>
  p_uid = 0
  p_pid = 2
  p_ppid = 1
  p_addr = 4263936
  p_size = 28672
> p_textp = 0xc01213b4 <g_ProcessManager+6420>
  p_stat = Process::SRUN
  p_flag = 1
  p_pri = 101
  p_cpu = 28
  p_nice = 0
  p_time = 0
  p_wchan = 0
  p_sig = 0
> p_ttyp = 0xc0124a80 <g_TTy>
  p_sigmap = 0
```

变量名称	含义	值
short p_uid	用户 ID	0
int p_pid	进程标识数	2
int p_ppid	父进程标识数	1
unsigned long p_addr	user 结构即 ppda 区的物理地址	4263936(0x411000)
unsigned int p_size	除共享正文段的长度，以字节单位	28672(0x7000)=28K
Text* p_textp	指向代码段 Text 结构的逻辑地址	0xc01213b4
ProcessState p_stat	进程调度状态	3=SRUN
int p_flag	进程标志位	1=SLOAD
int p_pri	进程优先数	101
int p_cpu	cpu 值，用于计算 p_pri	1c（28）
int p_nice	进程优先数微调参数	0
int p_time	进程在盘上(内存内)驻留时间	0
unsigned long p_wchan	进程睡眠原因	0

- Text 结构

```
p_textp = 0xc01213b4 <g_ProcessManager+64…
  x_daddr = 18232
  x_caddr = 4251648
  x_size = 12288
> x_iptr = 0xc0119d50 <g_InodeTable+304>
  x_count = 1
  x_ccount = 1
```

变量名称	含义	值
int x_daddr	代码段在盘交换区上的地址	18232(0x4738)
unsigned long x_caddr	代码段起始地址（物理地址）	4251648(0x40e000)
unsigned int x_size	代码段长度，以字节为单位	12288(0x3000)=12K
Inode* x_iptr	内存 inode 地址	0xc0119d50
Unsigned short x_count	共享正文段的进程数	1
Unsigned short x_ccount	共享该正文段且图像在内存的进程数	1

- **在 UNIX V6++中获取进程的代码段和可交换部分起始位置的逻辑地址和物理地址的方法：**

1. 从 user 结构中先找到代码段的起始地址 m_TextStartAddress(逻辑地址)；
2. 然后根据逻辑地址找到 Text 结构，根据 x_caddr 得到代码段的起始地址（物理地址）；
3. 从 user 结构中先找到 proc 结构的起始地址 u_procp(逻辑地址)；
4. 然后根据逻辑地址找到 Proc 结构，根据 p_addr 得到 PPDA 可交换部分起始位置（物理位置）；

（2）根据实验 4.1~4.2 的实验结果，绘制进程的相对虚实地址映射表。完成实验 4.3，获取完整的进程相对虚实地址映射表，补齐表 5，并验证自己给出的相对虚实地址映射表是否正确。

完整的相对虚实地址映射表如下：

页号	地址	值		
		高 20 位页框号	低 12 位标志位（u/s r/w p）	
0#	0xC0208000~0xC0208003			
	……			
1024#	0xC0208000~0xC0208003			
1025#	0xC0209004~0xC0209007	0x0	FFD（1111 1111 1101）	
1026#	0xC0209008~0xC020900B	0x1	FFD（1111 1111 1101）	代码段
1027#	0xC020900C~0xC020900F	0x2	FFD（1111 1111 1101）	
1028#	0xC0209010~0xC0209013	0x1	FFF（1111 1111 1111）	
1029#	0xC0209014~0xC0209017	0x2	FFF（1111 1111 1111）	
1030#	0xC0209018~0xC020901B	0x3	FFF（1111 1111 1111）	数据段
1031#	0xC020901C~0xC020901F	0x4	FFF（1111 1111 1111）	
1032#	0xC0209020~0xC0209023	0x5	FFF（1111 1111 1111）	
1033#	0xC020901C~0xC020901F	0x0	FFC（1111 1111 1100）	
	……	……	……	
2047#	0xC0209FFC~0xC0209FFF	0x6	FFF（0000 0000 0111）	堆栈段

根据实验结果补齐的表格与我设想的相同。实验 4.3 查看内存值的实验截图如下：

```
0xc0208ff0:     0x00000004      0x00000004      0x00000004      0x00000004
0xc0209000:     0x00000004      0x00000005      0x00001005      0x00002005
0xc0209010:     0x00001007      0x00002007      0x00003007      0x00004007
0xc0209020:     0x00005007      0x00000004      0x00000004      0x00000004
0xc0209030:     0x00000004      0x00000004      0x00000004      0x00000004
0xc0209040:     0x00000004      0x00000004      0x00000004      0x00000004
0xc0209050:     0x00000004      0x00000004      0x00000004      0x00000004
```

由上图可得代码段、数据段页框号。

```
0xc0209fd0:     0x00000004      0x00000004      0x00000004      0x00000004
0xc0209fe0:     0x00000004      0x00000004      0x00000004      0x00000004
0xc0209ff0:     0x00000004      0x00000004      0x00000004      0x00006007
0xc020a000:     0x6d6f632e      0x746e656d      0x00000150      0xffc00000
```

由上图可得到堆栈段的页框号。

（3）根据实验 4.1~4.2 的实验结果，绘制进程的 4 张物理页表。回答现运行进程的四张页表的逻辑地址是什么，再根据这个逻辑地址获取完整的进程物理页表，仿照表 5，自行绘制表格说明，并验证（2）中给出的 4 张物理页表是否正确。

绘制的 4 张物理页表如下：

0x200#物理页框--页目录【逻辑地址为 0xc0200000】

页表所在内存页框号	U/S	R/W	P
0x202	1	1	1
0x203	1	1	1
……			
0x201	0	1	1
……			

```
-exec x /100xw 0xc0200000
0xc0200000:     0x00202027      0x00203027      0x00000000      0x00000000
0xc0200010:     0x00000000      0x00000000      0x00000000      0x00000000
0xc0200020:     0x00000000      0x00000000      0x00000000      0x00000000
0xc0200030:     0x00000000      0x00000000      0x00000000      0x00000000
```

```
0xc0200be0:     0x00000000      0x00000000      0x00000000      0x00000000
0xc0200bf0:     0x00000000      0x00000000      0x00000000      0x00000000
0xc0200c00:     0x00201023      0x00000000      0x00000000      0x00000000
```

根据上述几幅图可以看出，实际内存单元中的值和课堂上所学一样。在 0#装入 0x202 号页框，1#装入 0x203 号页框，768#装入 0x201 号页框。

0x201#物理页框--内核页框【逻辑地址为 0xc0201000】

页表所在内存页框号	U/S	R/W	P
0x0	0	1	1
0x1	0	1	1
0x2	0	1	1
……			
0x411	0	1	1

```
0xc0201fc0:     0x003f0003      0x003f1003      0x003f2003      0x003f3003
0xc0201fd0:     0x003f4003      0x003f5003      0x003f6003      0x003f7003
0xc0201fe0:     0x003f8003      0x003f9003      0x003fa003      0x003fb003
0xc0201ff0:     0x003fc003      0x003fd003      0x003fe003      0x00411063
0xc0202000:     0x00000067      0x00001004      0x00002004      0x00003004
```

根据上述几幅图可以看出，实际内存单元中的值和课堂上所学一样。在 1023#写入的是 PPDA 区的起始地址所在页框（这与前文得到的 ppda 地址是一致的）。

0x202#物理页框--用户页表【逻辑地址为 0xc0202000】

页表所在内存页框号	U/S	R/W	P
/	/	/	/
/	/	/	/
/	/	/	/
……			
/	/	/	/

```
0xc0202000:     0x00000067      0x00001004      0x00002004      0x00003004
0xc0202010:     0x00004006      0x00005006      0x00006026      0x00007006
0xc0202020:     0x00008006      0x00009006      0x0000a006      0x0000b006
0xc0202030:     0x0000c006      0x0000d006      0x0000e006      0x0000f006
0xc0202040:     0x00010006      0x00011006      0x00012006      0x00013006
0xc0202050:     0x00014006      0x00015006      0x00016006      0x00017006
0xc0202060:     0x00018006      0x00019006      0x0001a006      0x0001b006
0xc0202070:     0x0001c006      0x0001d006      0x0001e006      0x0001f006
0xc0202080:     0x00020006      0x00021006      0x00022006      0x00023006
```

该页框为 0x202 页框，都是一些给编译器预留的值，无需深究含义。

0x203#物理页框--用户页表【逻辑地址为 0xc0203000】

页表所在内存页框号	U/S	R/W	P
/	/	/	/
0x40e	1	0	1
0x40f	1	0	1
0x410	1	0	1
0x412	1	1	1
0x413	1	1	1
0x414	1	1	1
0x415	1	1	1
0x416	1	1	1
……			
0x417	1	1	1

```
-exec x /100xw 0xc0203000
0xc0203000:     0x00400006      0x00040e065     0x00040f065     0x00410065
0xc0203010:     0x00412067      0x00413067      0x00414067      0x00415067
0xc0203020:     0x00416067      0x00416066      0x00417066      0x0040b006
0xc0203030:     0x0040c006      0x0040d006      0x0040e006      0x0040f006
0xc0203040:     0x00410006      0x00411006      0x00412006      0x00413006
```

```
0xc0203fd0:     0x007f4006      0x007f5006      0x007f6006      0x007f7006
0xc0203fe0:     0x007f8006      0x007f9006      0x007fa006      0x007fb006
0xc0203ff0:     0x007fc006      0x007fd006      0x007fe006      0x00417067
0xc0204000:     0x00000004      0x00000004      0x00000004      0x00000004
```

根据上述几幅图可以看出，实际内存单元中的值和课堂上所学一样。在 0#为编译器预留值，无需深究；1#开始是代码段的物理页框号，根据前文所述，代码段有 12K，共 3 张 4K 的页面，所以 0x40e 到 0x410 是代码段；数据段有 20K，共 5 张 4K 页面，所以 0x412 到 0x416 都是数据段；最后一个为堆栈段，堆栈段有 4K，共一张 4K 页面，所以 0x417 是堆栈段。这与前文得到的值都是一致的。验证了上述我绘制页表的正确性。

PS：与课上不同的是，数据段和堆栈段之间的值并非全 0，而是有数值，并且递增。